

```
In [5]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

%matplotlib inline
```

1) Basic Pandas

```
In [6]: sales = pd.read_csv(
    'sales_data.csv',
    parse_dates=['Date'])
```

parse_dates=['Date']: A parameter that instructs pandas to parse the values in the 'Date' column as actual timestamps (datetime64) rather than plain text strings. This is essential for time-series operations, allowing you to easily filter by date ranges, resample data, or plot trends over time.

```
In [7]: sales.head()
```

Out[7]:

	Date	Day	Month	Year	Customer_Age	Age_Group	Customer_Gender	Country	
0	2013-11-26	26	November	2013	19	Youth (<25)	M	Canada	C
1	2015-11-26	26	November	2015	19	Youth (<25)	M	Canada	C
2	2014-03-23	23	March	2014	49	Adults (35-64)	M	Australia	
3	2016-03-23	23	March	2016	49	Adults (35-64)	M	Australia	
4	2014-05-15	15	May	2014	47	Adults (35-64)	F	Australia	

```
In [8]: sales.shape
```

Out[8]: (113036, 18)

```
In [9]: sales.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 113036 entries, 0 to 113035
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Date                   113036 non-null  datetime64[us]
1   Day                    113036 non-null  int64
2   Month                  113036 non-null  str
3   Year                   113036 non-null  int64
4   Customer_Age           113036 non-null  int64
5   Age_Group              113036 non-null  str
6   Customer_Gender        113036 non-null  str
7   Country                113036 non-null  str
8   State                  113036 non-null  str
9   Product_Category       113036 non-null  str
10  Sub_Category           113036 non-null  str
11  Product                113036 non-null  str
12  Order_Quantity          113036 non-null  int64
13  Unit_Cost               113036 non-null  int64
14  Unit_Price              113036 non-null  int64
15  Profit                  113036 non-null  int64
16  Cost                    113036 non-null  int64
17  Revenue                 113036 non-null  int64
dtypes: datetime64[us](1), int64(9), str(8)
memory usage: 24.6 MB
```

In [10]: sales.describe()

Out[10]:

	Date	Day	Year	Customer_Age	Order_Quantity	
count	113036	113036.000000	113036.000000	113036.000000	113036.000000	113036.000000
mean	2014-11-23 12:14:55.063519	15.665753	2014.401739	35.919212	11.901660	20.000000
min	2011-01-01 00:00:00	1.000000	2011.000000	17.000000	1.000000	1.000000
25%	2013-12-22 00:00:00	8.000000	2013.000000	28.000000	2.000000	2.000000
50%	2014-06-27 00:00:00	16.000000	2014.000000	35.000000	10.000000	10.000000
75%	2016-01-09 00:00:00	23.000000	2016.000000	43.000000	20.000000	20.000000
max	2016-07-31 00:00:00	31.000000	2016.000000	87.000000	32.000000	21.000000
std	NaN	8.781567	1.272510	11.021936	9.561857	5.000000

2) Numerical analysis and visualization

We'll analyze the `Unit\_Cost` column:

```
In [11]: sales['Unit_Cost'].describe()
```

```
Out[11]: count    113036.000000
         mean      267.296366
         std       549.835483
         min        1.000000
         25%        2.000000
         50%        9.000000
         75%       42.000000
         max      2171.000000
         Name: Unit_Cost, dtype: float64
```

```
In [12]: sales['Unit_Cost'].mean()
```

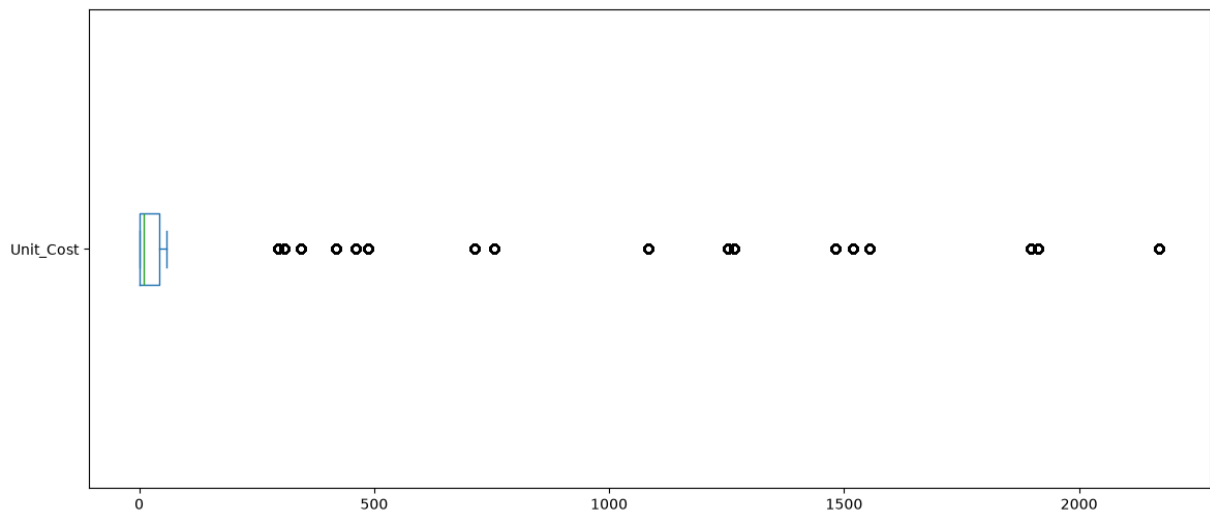
```
Out[12]: np.float64(267.296365759581)
```

```
In [13]: sales['Unit_Cost'].median()
```

```
Out[13]: np.float64(9.0)
```

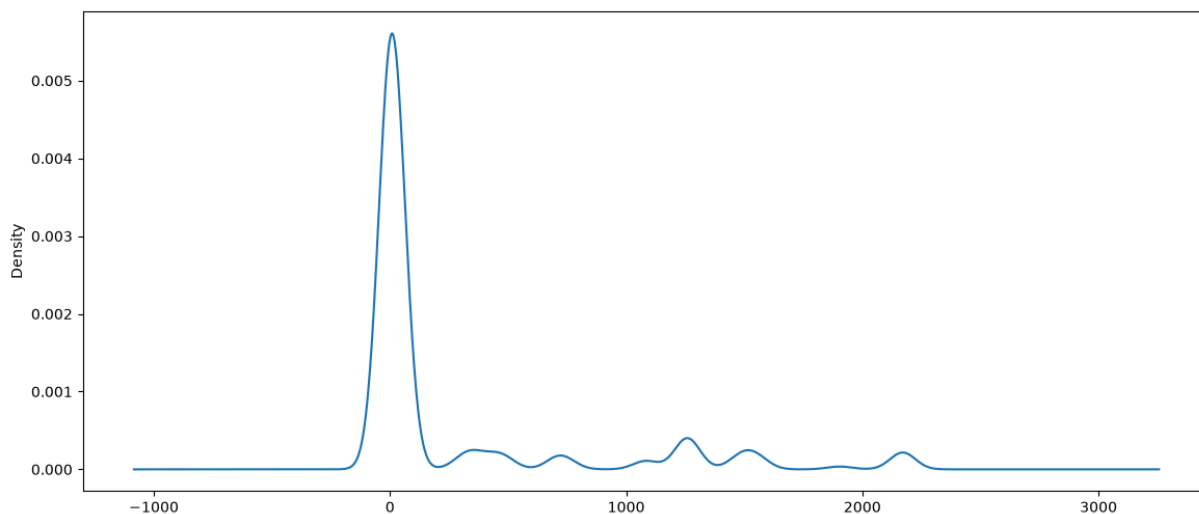
```
In [14]: sales['Unit_Cost'].plot(kind = 'box', vert = False, figsize=(14,6))
```

```
Out[14]: <Axes: >
```



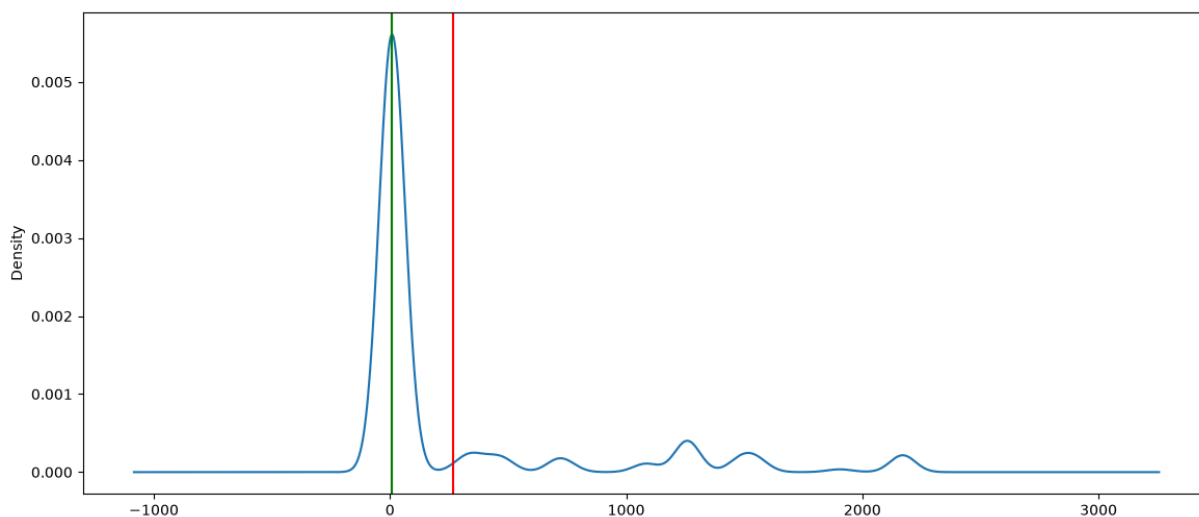
```
In [15]: sales['Unit_Cost'].plot(kind='density', figsize=(14,6)) # kde
```

```
Out[15]: <Axes: ylabel='Density'>
```



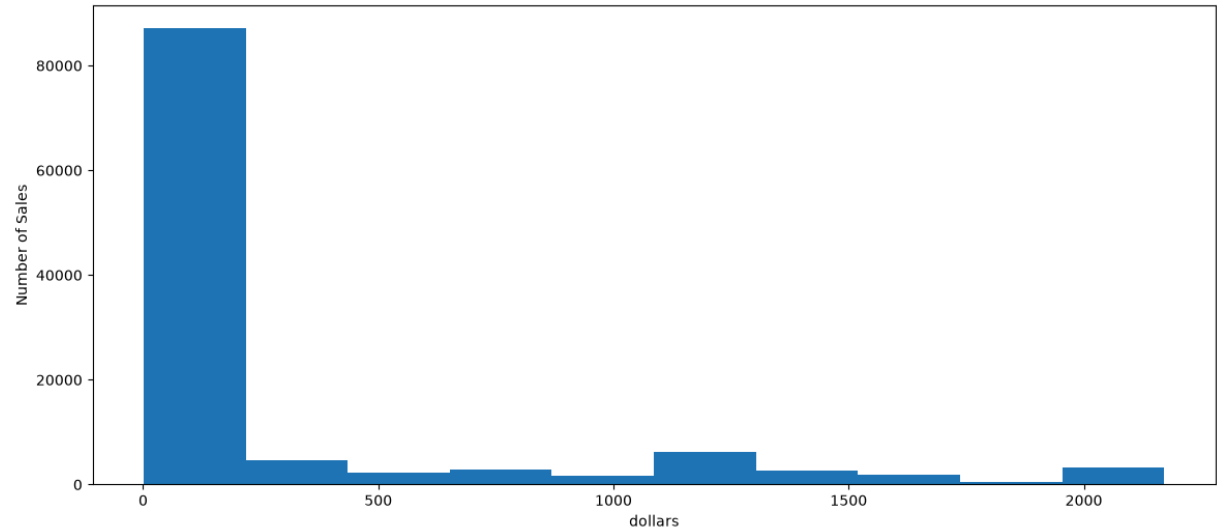
```
In [16]: ax = sales['Unit_Cost'].plot(kind='density', figsize=(14,6)) # kde  
ax.axvline(sales['Unit_Cost'].mean(), color='red')  
ax.axvline(sales['Unit_Cost'].median(), color='green')
```

Out[16]: <matplotlib.lines.Line2D at 0x147b4cf9160>



```
In [17]: ax = sales['Unit_Cost'].plot(kind='hist', figsize=(14,6))  
ax.set_ylabel('Number of Sales')  
ax.set_xlabel('dollars')
```

Out[17]: Text(0.5, 0, 'dollars')



3) Categorical analysis and visualization

We'll analyze the Age_Group column:

```
In [21]: sales.head()
```

Out[21]:

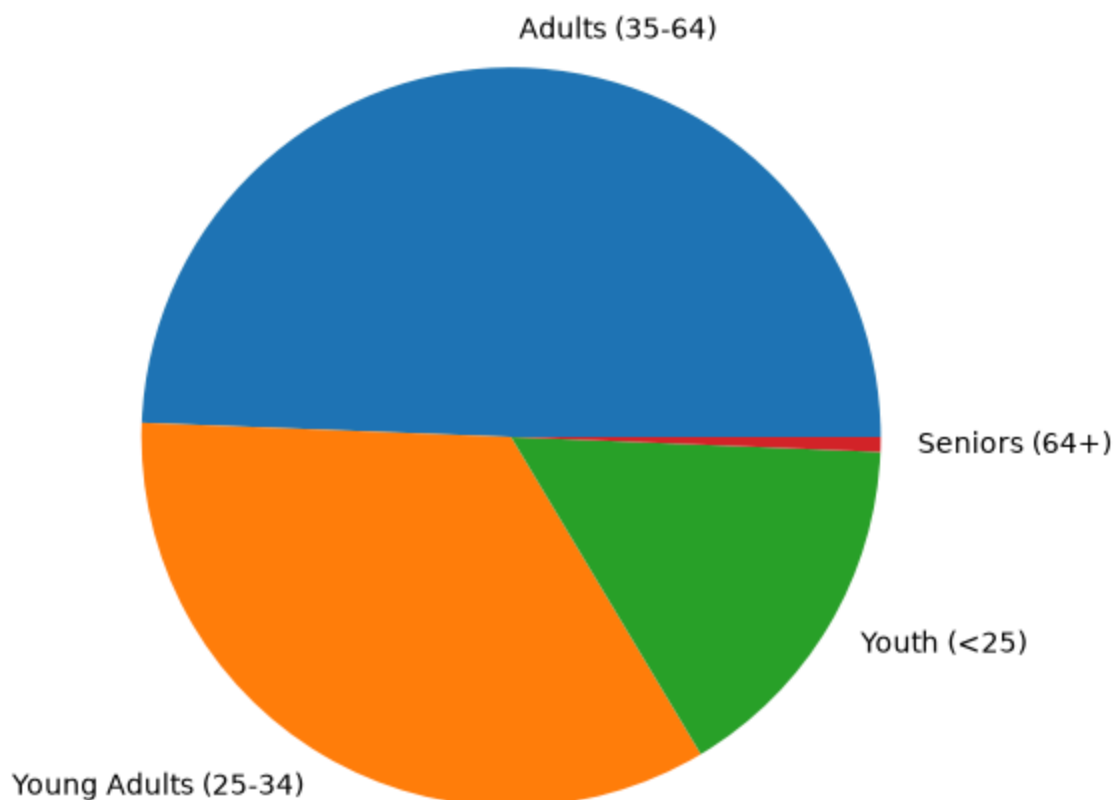
	Date	Day	Month	Year	Customer_Age	Age_Group	Customer_Gender	Country	
0	2013-11-26	26	November	2013	19	Youth (<25)	M	Canada	C
1	2015-11-26	26	November	2015	19	Youth (<25)	M	Canada	C
2	2014-03-23	23	March	2014	49	Adults (35-64)	M	Australia	
3	2016-03-23	23	March	2016	49	Adults (35-64)	M	Australia	
4	2014-05-15	15	May	2014	47	Adults (35-64)	F	Australia	

```
In [22]: sales['Age_Group'].value_counts()
```

```
Out[22]: Age_Group
Adults (35-64)      55824
Young Adults (25-34) 38654
Youth (<25)         17828
Seniors (64+)        730
Name: count, dtype: int64
```

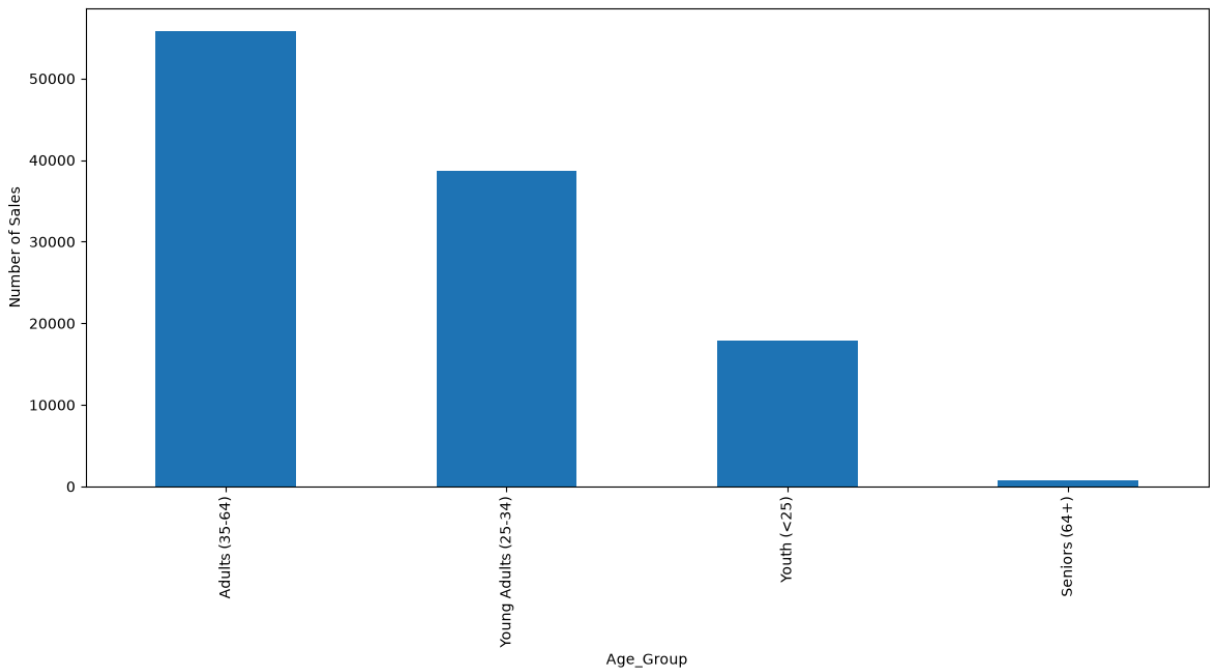
```
In [23]: sales['Age_Group'].value_counts().plot(kind='pie', figsize=(6,6))
```

```
Out[23]: <Axes: >
```



```
In [24]: ax = sales['Age_Group'].value_counts().plot(kind='bar', figsize=(14,6))
ax.set_ylabel('Number of Sales')    # just naming label on y-axis
```

```
Out[24]: Text(0, 0.5, 'Number of Sales')
```



4) Relationship between the columns?

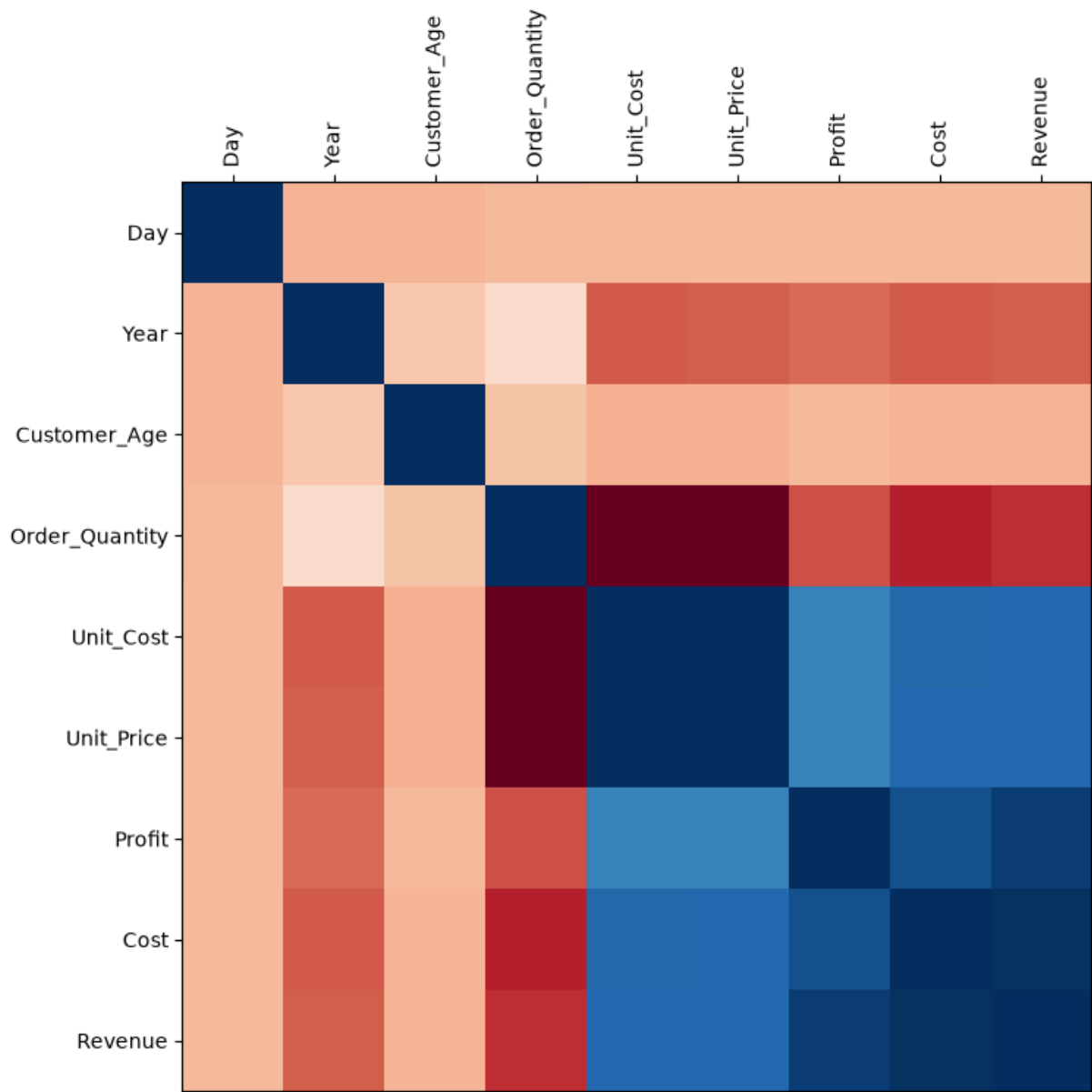
Can we find any significant relationship?

```
In [25]: corr = sales.corr(numeric_only=True)
corr
```

```
Out[25]:
```

	Day	Year	Customer_Age	Order_Quantity	Unit_Cost	Unit_Price
Day	1.000000	-0.007635	-0.014296	-0.002412	0.003133	0.003207
Year	-0.007635	1.000000	0.040994	0.123169	-0.217575	-0.213673
Customer_Age	-0.014296	0.040994	1.000000	0.026887	-0.021374	-0.020262
Order_Quantity	-0.002412	0.123169	0.026887	1.000000	-0.515835	-0.515925
Unit_Cost	0.003133	-0.217575	-0.021374	-0.515835	1.000000	0.997894
Unit_Price	0.003207	-0.213673	-0.020262	-0.515925	0.997894	1.000000
Profit	0.004623	-0.181525	0.004319	-0.238863	0.741020	0.749870
Cost	0.003329	-0.215604	-0.016013	-0.340382	0.829869	0.826301
Revenue	0.003853	-0.208673	-0.009326	-0.312895	0.817865	0.818522

```
In [26]: fig = plt.figure(figsize=(8,8))
plt.matshow(corr, cmap='RdBu', fignum=fig.number)
plt.xticks(range(len(corr.columns)), corr.columns, rotation='vertical');
plt.yticks(range(len(corr.columns)), corr.columns);
```

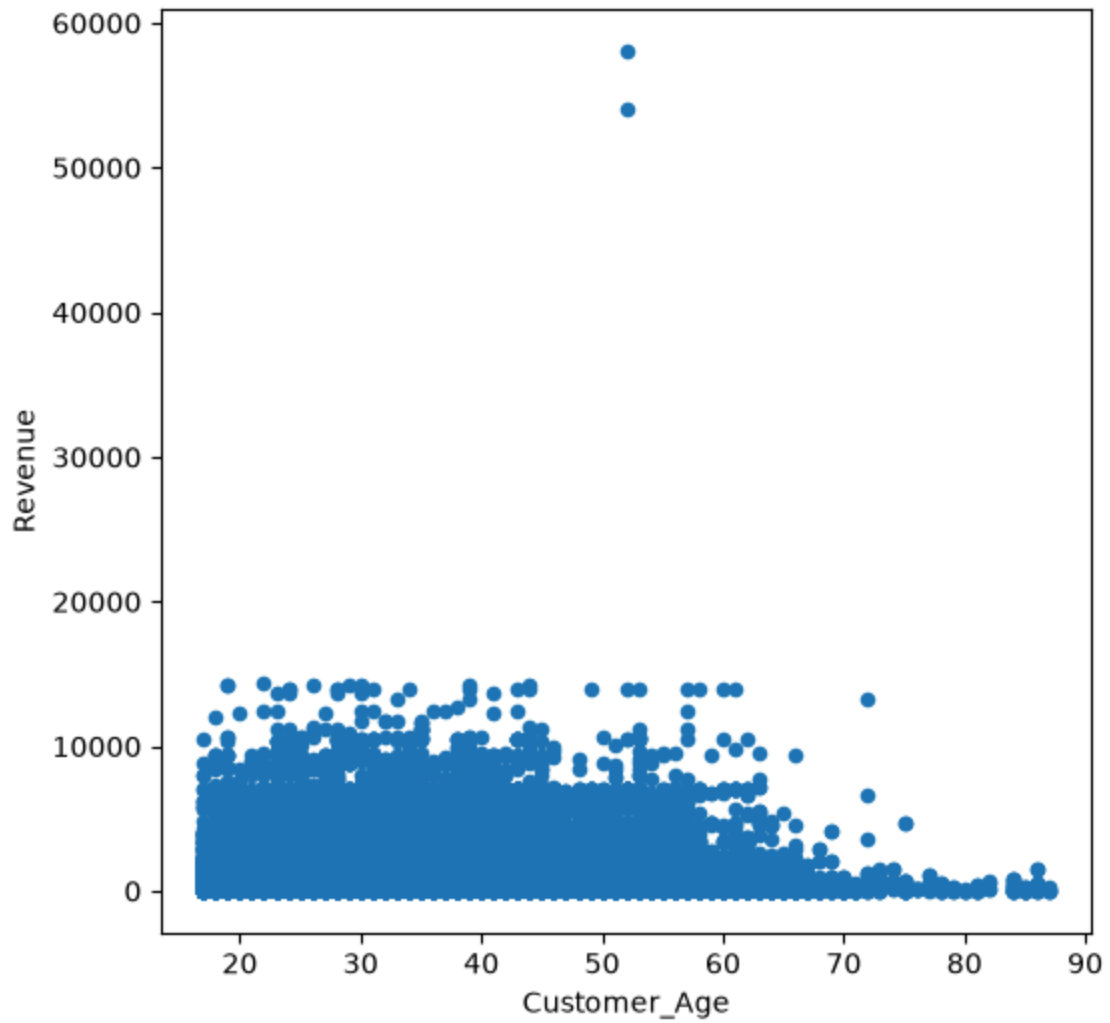


```
In [27]: sales['Revenue'].describe()
```

```
Out[27]: count    113036.000000
         mean       754.370360
         std       1309.094674
         min         2.000000
         25%        63.000000
         50%       223.000000
         75%       800.000000
         max      58074.000000
         Name: Revenue, dtype: float64
```

```
In [28]: sales.plot(kind='scatter', x='Customer_Age', y='Revenue', figsize=(6,6))
```

```
Out[28]: <Axes: xlabel='Customer_Age', ylabel='Revenue'>
```



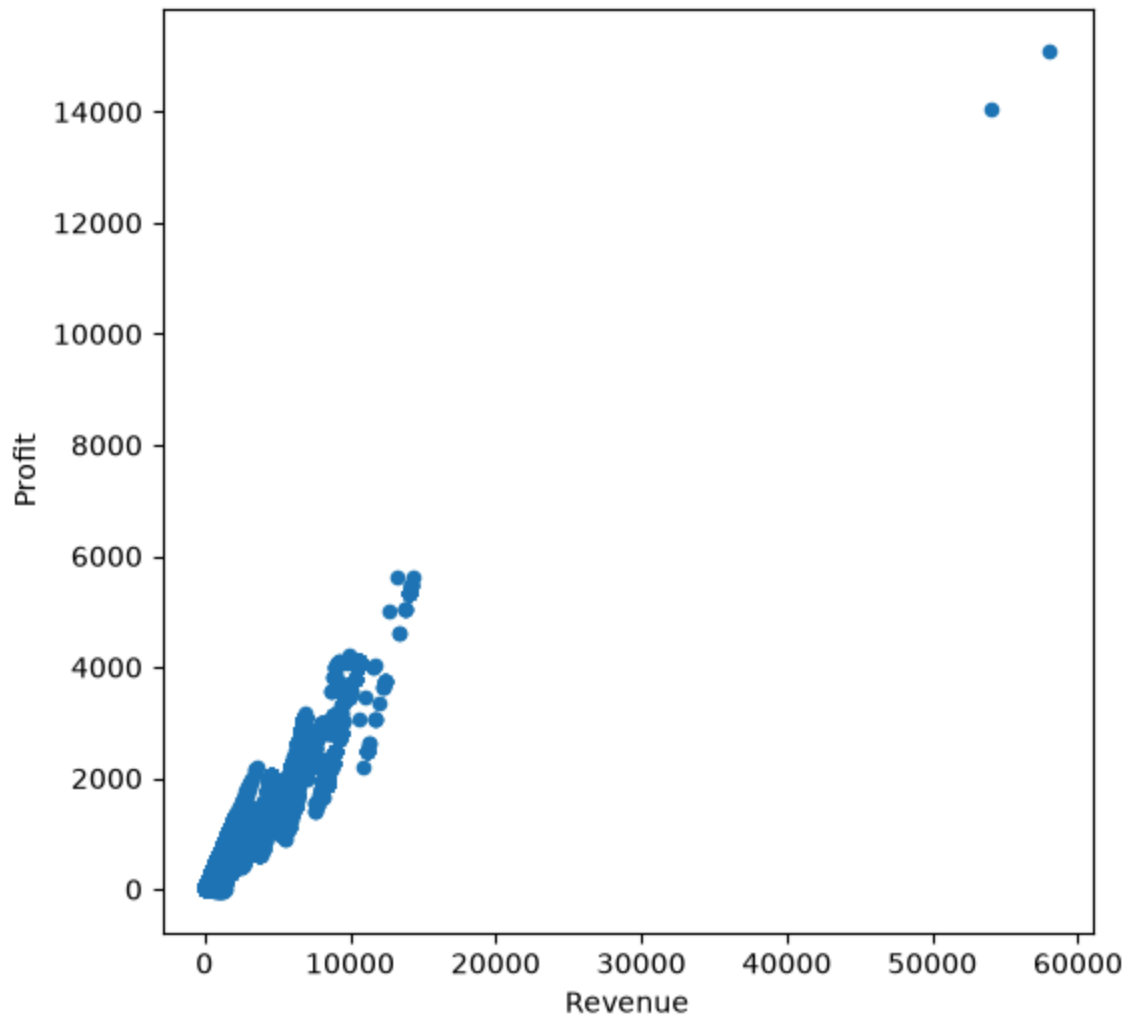
```
In [29]: sales.head()
```

Out[29]:

	Date	Day	Month	Year	Customer_Age	Age_Group	Customer_Gender	Country	
0	2013-11-26	26	November	2013	19	Youth (<25)	M	Canada	C
1	2015-11-26	26	November	2015	19	Youth (<25)	M	Canada	C
2	2014-03-23	23	March	2014	49	Adults (35-64)	M	Australia	
3	2016-03-23	23	March	2016	49	Adults (35-64)	M	Australia	
4	2014-05-15	15	May	2014	47	Adults (35-64)	F	Australia	

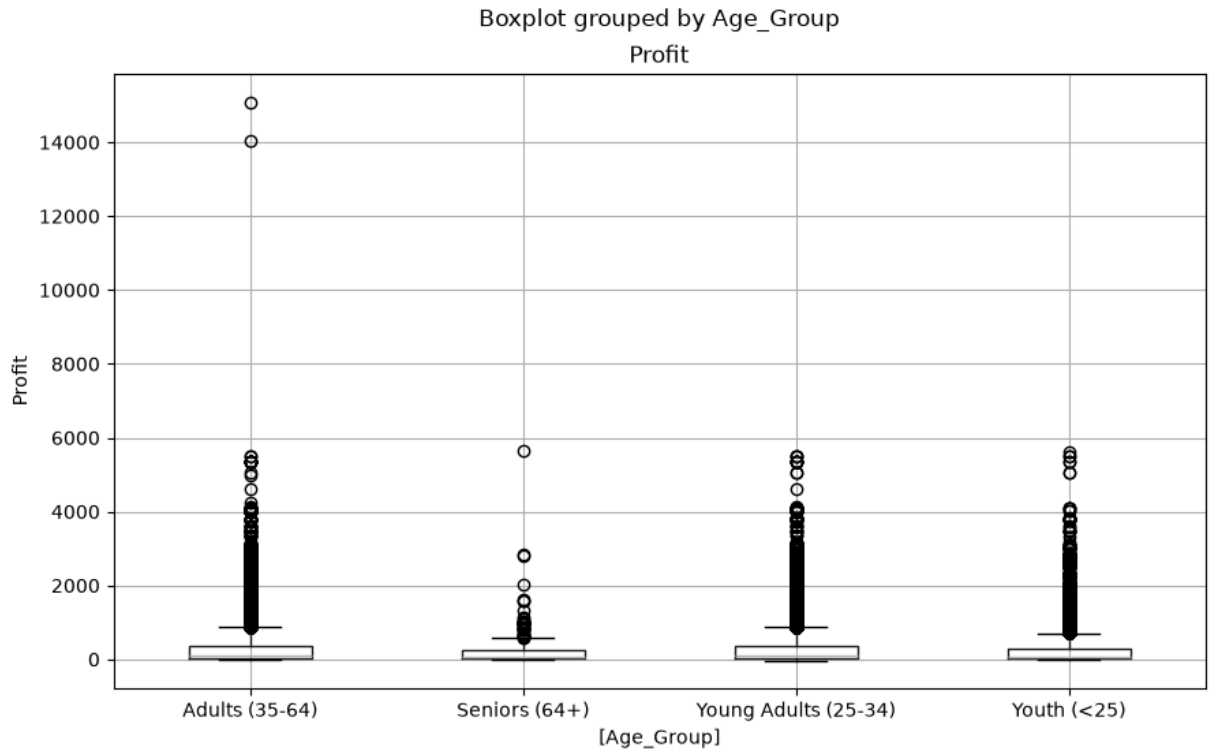
In [30]: sales.plot(kind='scatter', x='Revenue', y='Profit', figsize=(6,6))

Out[30]: <Axes: xlabel='Revenue', ylabel='Profit'>



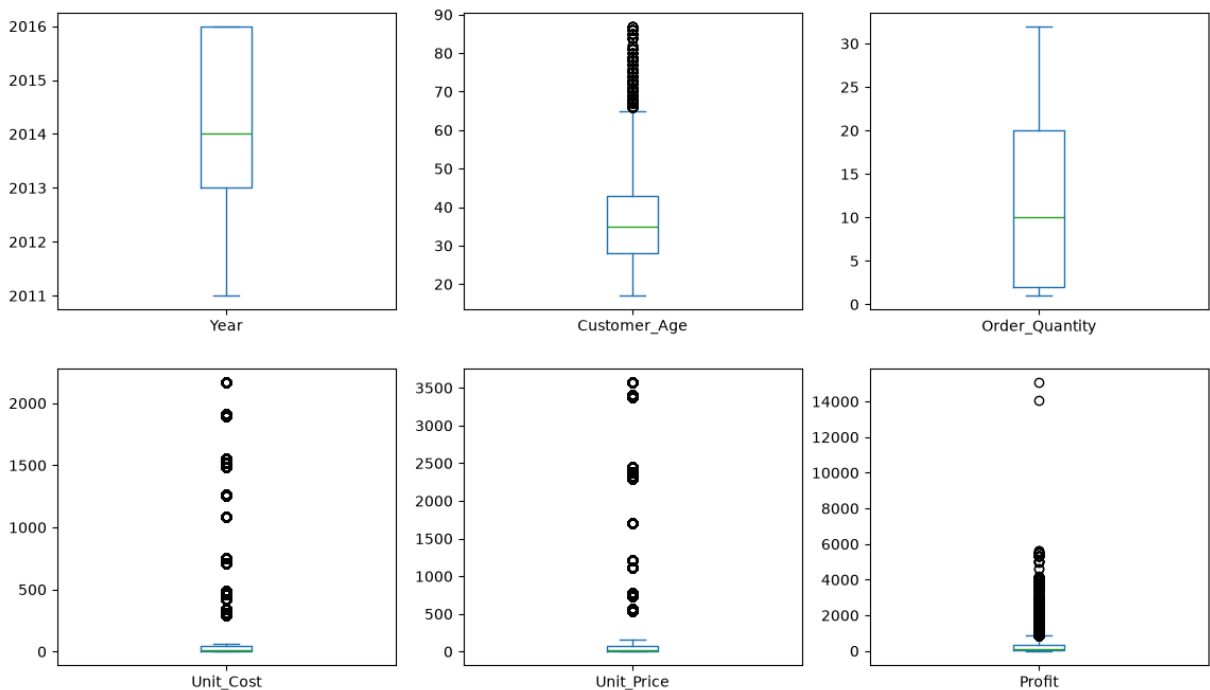
```
In [31]: ax = sales[['Profit', 'Age_Group']].boxplot(by='Age_Group', figsize=(10,6))  
ax.set_ylabel('Profit')
```

```
Out[31]: Text(0, 0.5, 'Profit')
```



```
In [32]: boxplot_cols = ['Year', 'Customer_Age', 'Order_Quantity', 'Unit_Cost', 'Unit_Price', 'Profit']
sales[boxplot_cols].plot(kind='box', subplots=True, layout=(2,3), figsize=(14,8))
```

```
Out[32]: Year          Axes(0.125,0.53;0.227941x0.35)
Customer_Age        Axes(0.398529,0.53;0.227941x0.35)
Order_Quantity       Axes(0.672059,0.53;0.227941x0.35)
Unit_Cost            Axes(0.125,0.11;0.227941x0.35)
Unit_Price           Axes(0.398529,0.11;0.227941x0.35)
Profit              Axes(0.672059,0.11;0.227941x0.35)
dtype: object
```



5) Column wrangling

We can also create new columns or modify existing ones.

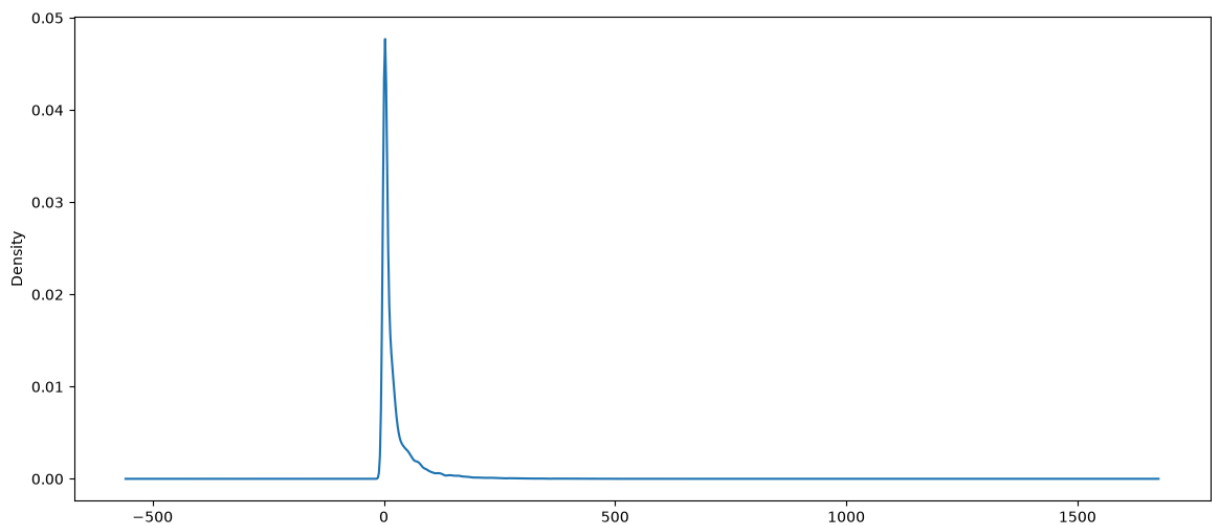
Add and calculate a new `Revenue_per_Age` column

```
In [33]: sales['Revenue_per_Age'] = sales['Revenue'] / sales['Customer_Age']  
  
sales['Revenue_per_Age'].head()
```

```
Out[33]: 0    50.000000  
        1    50.000000  
        2    49.000000  
        3    42.612245  
        4     8.893617  
        Name: Revenue_per_Age, dtype: float64
```

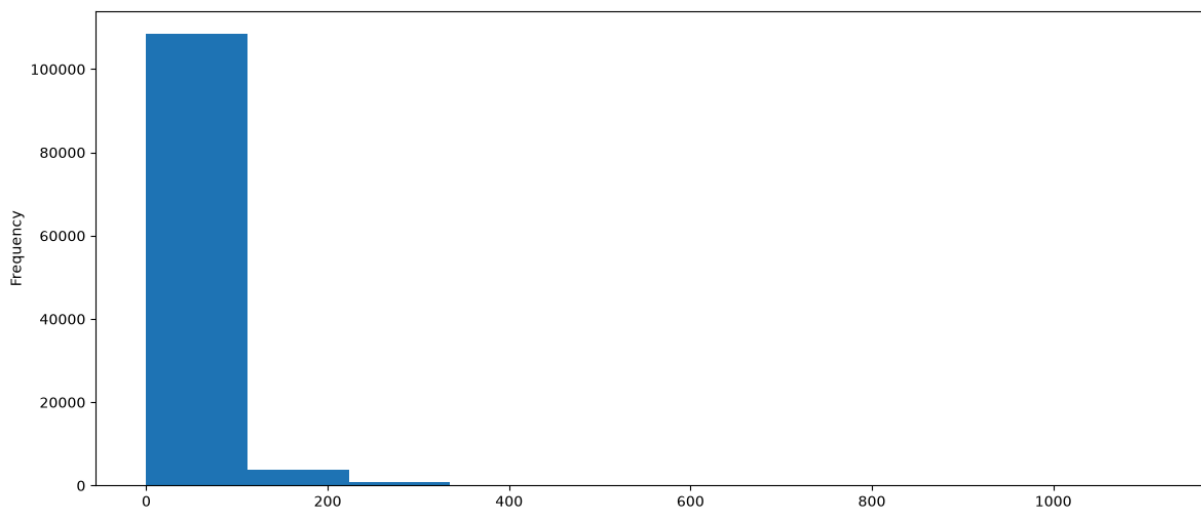
```
In [34]: sales['Revenue_per_Age'].plot(kind='density', figsize=(14,6))
```

```
Out[34]: <Axes: ylabel='Density'>
```



```
In [35]: sales['Revenue_per_Age'].plot(kind='hist', figsize=(14,6))
```

```
Out[35]: <Axes: ylabel='Frequency'>
```



Add and calculate a new `Calculated_Cost` column

Use this formula

$\text{Calculated_Cost} = \text{Order_Quantity} * \text{Unit_Cost}$

```
In [36]: sales['Calculated_Cost'] = sales['Order_Quantity'] * sales['Unit_Cost']
         sales['Calculated_Cost'].head()
```

```
Out[36]: 0      360
         1      360
         2     1035
         3      900
         4      180
         Name: Calculated_Cost, dtype: int64
```

```
In [37]: (sales['Calculated_Cost'] != sales['Cost']).sum()
```

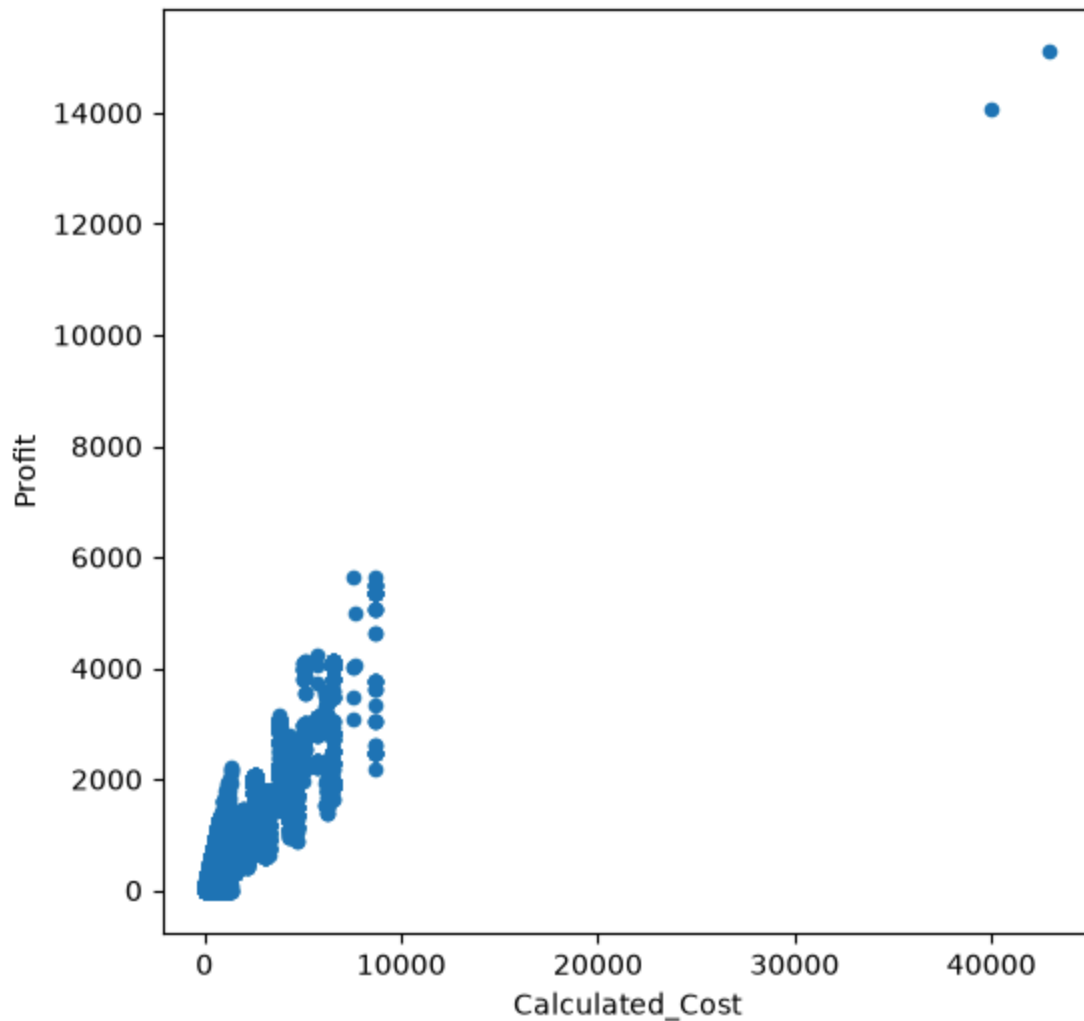
```
Out[37]: np.int64(0)
```

```
In [38]: sales.head()
```

Out[38]:

	Date	Day	Month	Year	Customer_Age	Age_Group	Customer_Gender	Country	
0	2013-11-26	26	November	2013	19	Youth (<25)	M	Canada	C
1	2015-11-26	26	November	2015	19	Youth (<25)	M	Canada	C
2	2014-03-23	23	March	2014	49	Adults (35-64)	M	Australia	
3	2016-03-23	23	March	2016	49	Adults (35-64)	M	Australia	
4	2014-05-15	15	May	2014	47	Adults (35-64)	F	Australia	

In [39]: `sales.plot(kind='scatter', x='Calculated_Cost', y='Profit', figsize=(6,6))`Out[39]: `<Axes: xlabel='Calculated_Cost', ylabel='Profit'>`



6) Add and calculate a new `Calculated\_Revenue` column

Use this formula

$\text{Calculated_Revenue} = \text{Cost} + \text{Profit}$

```
In [40]: sales['Calculated_Revenue'] = sales['Cost'] + sales['Profit']  
  
sales['Calculated_Revenue'].head()
```

```
Out[40]: 0    950  
         1    950  
         2   2401  
         3   2088  
         4    418  
         Name: Calculated_Revenue, dtype: int64
```

```
In [41]: (sales['Calculated_Revenue'] != sales['Revenue']).sum()
```

Out[41]: np.int64(0)

sales['Calculated_Revenue'] != sales['Revenue']: Compares the two columns row by row, returning a boolean Series of True (where the values differ) and False (where they match).

.sum(): Treats True as 1 and False as 0, summing them up to give the total count of discrepancies between the two columns.

```
In [42]: sales.head()
```

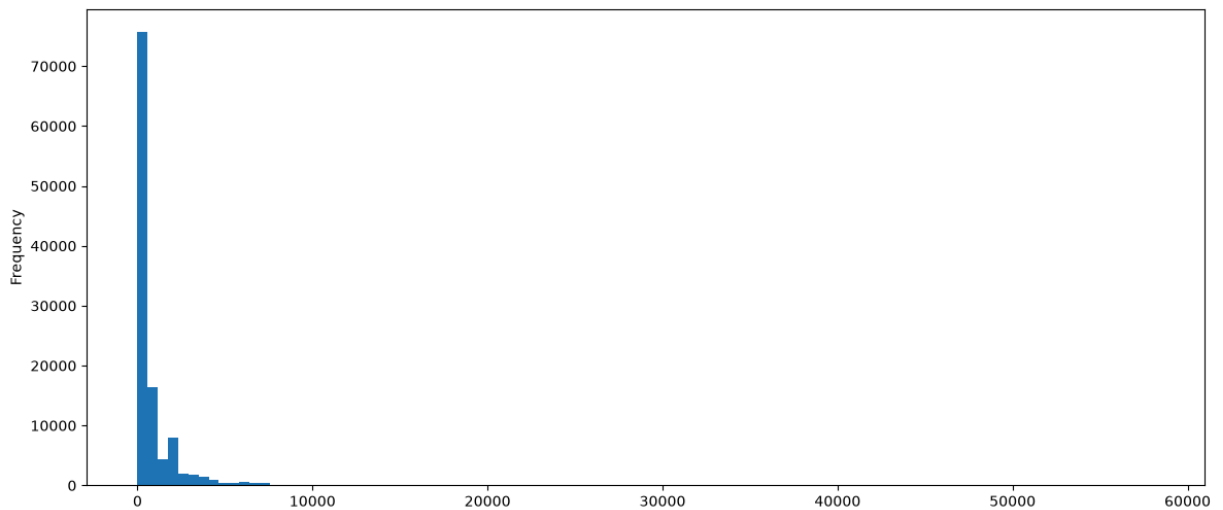
Out[42]:

	Date	Day	Month	Year	Customer_Age	Age_Group	Customer_Gender	Country	
0	2013-11-26	26	November	2013	19	Youth (<25)	M	Canada	C
1	2015-11-26	26	November	2015	19	Youth (<25)	M	Canada	C
2	2014-03-23	23	March	2014	49	Adults (35-64)	M	Australia	
3	2016-03-23	23	March	2016	49	Adults (35-64)	M	Australia	
4	2014-05-15	15	May	2014	47	Adults (35-64)	F	Australia	

5 rows × 21 columns

```
In [43]: sales['Revenue'].plot(kind='hist', bins=100, figsize=(14,6))
```

Out[43]: <Axes: ylabel='Frequency'>



7) Modify all `Unit\_Price` values adding 3% tax to them

PRECAUTION - Created sales dataframe again as running 10% code altering real value

```
In [166...] sales = pd.read_csv(
    'sales_data.csv',
    parse_dates=['Date'])          #needed to reload sales dataframe because
```

```
In [167...] sales['Unit_Price'].head()
```

```
Out[167...] 0    120
            1    120
            2    120
            3    120
            4    120
            Name: Unit_Price, dtype: int64
```

```
In [168...] #sales['Unit_Price'] = sales['Unit_Price'] * 1.03

            sales['Unit_Price'] *= 1.03
```

```
In [169...] sales['Unit_Price'].head()
```

```
Out[169...] 0    123.6
            1    123.6
            2    123.6
            3    123.6
            4    123.6
            Name: Unit_Price, dtype: float64
```

8) Selection & Indexing:

Get all the sales made in the state of **Kentucky**

In [49]:

sales.loc[sales['State'] == 'Kentucky']#Loc means by index Kentucky

Out[49]:

	Date	Day	Month	Year	Customer_Age	Age_Group	Customer_Gender	Count
156	2013-11-04	4	November	2013	40	Adults (35-64)	M	United States
157	2015-11-04	4	November	2015	40	Adults (35-64)	M	United States
23826	2014-04-16	16	April	2014	40	Adults (35-64)	M	United States
23827	2016-04-16	16	April	2016	40	Adults (35-64)	M	United States
31446	2014-04-16	16	April	2014	40	Adults (35-64)	M	United States
31447	2016-04-16	16	April	2016	40	Adults (35-64)	M	United States
79670	2014-04-16	16	April	2014	40	Adults (35-64)	M	United States
79671	2014-04-16	16	April	2014	40	Adults (35-64)	M	United States
79672	2016-04-16	16	April	2016	40	Adults (35-64)	M	United States
79673	2016-04-16	16	April	2016	40	Adults (35-64)	M	United States

10 rows × 21 columns



```
In [67]: sales.loc[sales['State'] == 'Kentucky'].shape
```

```
Out[67]: (10, 21)
```

Get the mean revenue of the Adults (35-64) sales group

```
In [74]: sales.loc[sales['Age_Group'] == 'Adults (35-64)', 'Revenue'] # seeing
```

```
Out[74]: 2      2401
          3      2088
          4       418
          5       522
          6       379
          ...
113029    123
113030    369
113031    184
113034    1260
113035    1207
Name: Revenue, Length: 55824, dtype: int64
```

```
In [75]: sales.loc[sales['Age_Group'] == 'Adults (35-64)', 'Revenue'].mean() # right
```

```
Out[75]: np.float64(762.8287654055604)
```

How many records belong to Age Group Youth (<25) or Adults (35-64) ?

```
In [65]: sales.loc[(sales['Age_Group'] == 'Youth (<25)') | (sales['Age_Group'] == 'Adults (35-64)')]
```

```
Out[65]: (73652, 21)
```

```
In [68]: sales.loc[(sales['Age_Group'] == 'Youth (<25)') | (sales['Age_Group'] == 'Adults (35-64)')].shape
```

```
Out[68]: 73652
```

It returns the total count of rows that match your filtering conditions (i.e., the number of records where the age group is either 'Youth (<25)' or 'Adults (35-64)').

`sales.loc[...]`: Filters the DataFrame to include only the rows where the specified conditions are met.

`.shape`: Returns a tuple representing the dimensions of the filtered DataFrame in the format (number_of_rows, number_of_columns).

`[0]`: Extracts the first element of that tuple, isolating just the number of rows.

Get the mean revenue of the sales group Adults (35-64) in United States

```
In [101...] sales.loc[(sales['Age_Group'] == 'Adults (35-64)') & (sales['Country'] == 'United S
```

```
Out[101...] 22      929.0
            23      722.0
            38     1176.0
            39     1176.0
            40     1294.0
            ...
            112959   499.0
            112986   878.0
            112987   878.0
            112994  1756.0
            112995  1568.0
            Name: Revenue, Length: 21960, dtype: float64
```

```
In [102...] sales.loc[(sales['Age_Group'] == 'Adults (35-64)') & (sales['Country'] == 'United S
```

```
Out[102...] np.float64(726.7260473588342)
```

Increase the revenue by 10% to every sale made in France

PRECAUTION - Created sales dataframe again as running 10% code altering real value

```
In [140...] sales = pd.read_csv(
    'sales_data.csv',
    parse_dates=['Date'])          #needed to reload sales dataframe because
```

```
In [141...] sales.loc[sales['Country'] == 'France', 'Revenue'].head()
```

```
Out[141...] 50      787
            51      787
            52     2957
            53     2851
            60      626
            Name: Revenue, dtype: int64
```

```
In [142...] sales['Revenue'] = sales['Revenue'].astype(float)          # needed to cast whole i
sales.loc[sales['Country'] == 'France', 'Revenue'] *= 1.1
```

```
In [143...] sales.loc[sales['Country'] == 'France', 'Revenue'].head()
```

```
Out[143...  50      865.7
           51      865.7
           52     3252.7
           53     3136.1
           60      688.6
           Name: Revenue, dtype: float64
```

```
In [ ]:
```

```
In [ ]:
```